

Multi-Hop VC Due Diligence System

Built by Samhitha Kanth

Project Overview

This project is an **AI-powered Venture Capital Due Diligence System** designed to analyze startup technical claims and detect hidden risks.

The system takes three major inputs:

- Startup pitch deck / whitepaper
- Startup codebase (GitHub repository)
- Patent documents

It automatically identifies:

- False proprietary claims
- Hidden dependency risks
- Patent overlap / IP risks
- Weak technical moat
- Legal discrepancies

Finally, it generates **due diligence questions** using:

- graph reasoning
 - semantic retrieval
 - LLM-based RAG
-

High-Level Architecture

```
INPUT SOURCES
|
├─ whitepaper_parser.py
├─ github_parser.py
└─ patent_parser.py
|
▼
knowledge_fusion.py
(NetworkX heterogeneous graph)
```

```

|
|— Claim nodes
|— Dependency nodes
|— Patent nodes
|
▼
Semantic Layer
|
|— faiss_indexer.py
|— retriever.py
|
▼
Reasoning Layer
|
|— mhqg_engine.py
|— legal_analyzer.py
|
▼
LLM Layer
|
|— llm_mhqg.py (Ollama + Llama3)
|
▼
OUTPUT
|
|— due_diligence_questions.json

```

Folder Structure

```

CCBD/
|
|— data/
|   |— raw/
|   |   |— patents/
|   |   |— pitch_decks/
|   |   └— repositories/
|   └— processed/
|       |— expense_ninja_pitch_parsed.json
|       |— dependency_map.json
|       |— import_graph.json
|       |— import_graph_analysis.json
|       └— codebase_knowledge.json

```

```
|
|   ├── knowledge_base.json
|   ├── patent_mock_triples.json
|   ├── fused_knowledge_graph.json
|   ├── vector.index
|   ├── vector_metadata.json
|   ├── due_diligence_questions.json
|   └── llm_due_diligence_output.json
|
|── src/
|   |
|   ├── extractors/
|   |   ├── whitepaper_parser.py
|   |   ├── github_parser.py
|   |   └── patent_parser.py
|   |
|   ├── graph/
|   |   └── knowledge_fusion.py
|   |
|   ├── vector/
|   |   ├── faiss_indexer.py
|   |   └── retriever.py
|   |
|   ├── legal/
|   |   └── legal_analyzer.py
|   |
|   └── reasoning/
|       ├── mhqg_engine.py
|       └── llm_mhqg.py
|
|── tests/
|
|── requirements.txt
|
|── README.md
```

File-by-File Explanation

1. whitepaper_parser.py

Purpose

Extract technical claims from startup pitch decks.

Input

PDF pitch deck

Output

expense_ninja_pitch_parsed.json

Features

- PDF extraction using:
 - pdflumber
 - pypdf fallback
- OCR using Tesseract
- Technical claim detection across:
 - security
 - performance
 - architecture
 - protocol
 - data
- Buzzword resolution

Example:

military-grade encryption

Converted into:

AES-256 / cryptographic security

- Regex extraction for:
 - SHA
 - ECDSA
 - TLS
 - JWT
- Numeric claim extraction:
 - TPS
 - latency
 - throughput

2. github_parser.py

Purpose

Analyze startup source code.

Output

```
dependency_map.json  
import_graph.json  
codebase_knowledge.json
```

Mode A — Dependency Mapping

Parses:

- requirements.txt
- pyproject.toml
- package.json
- Pipfile

Extracts libraries such as:

```
bcrypt  
cryptography  
PyJWT  
networkx
```

Mode B — Import Graph Analysis

Uses:

- Python AST
- NetworkX graph

Detects:

- Hub modules
- Orphan modules
- Circular imports

Example graph:

moduleA → moduleB → moduleC

3. patent_parser.py

Purpose

Convert patent text into structured knowledge.

Input

Patent text / patent abstract

Output

knowledge_base.json

Extracts triples:

head → relationship → tail

Example:

```
cryptographic hashing module
    encrypts
user password
```

Stored as:

```
{
  "head": "cryptographic hashing module",
  "relationship": "encrypts",
  "tail": "user password"
}
```

Also extracts metadata:

- patent ID

- status
 - assignee
 - jurisdiction
 - legal risk flags
-

4. knowledge_fusion.py

Purpose

Fuse all extracted knowledge into one graph.

Combines:

- Marketing claims
- Software dependencies
- Patent concepts

Uses:

NetworkX DiGraph

Node Types

Marketing_Claim

Example:

Proprietary military-grade encryption

Software_Dependency

Example:

bcrypt

Patent_Concept

Example:

cryptographic hashing module

Edge Types

IMPLEMENTED_BY

Claim → Dependency

Example:

Encryption claim → bcrypt

SIMILAR_TO

Dependency → Patent Concept

Example:

bcrypt → hashing module

Output

fused_knowledge_graph.json

Also exports GraphML for visualization.

5. faiss_indexer.py

Purpose

Create vector database.

Uses:

- Sentence Transformers
- FAISS
- MiniLM embeddings

Each node becomes a vector.

Example:

```
bcrypt password hashing library
```

Becomes:

```
384-dimensional embedding vector
```

Stored in:

```
vector.index  
vector_metadata.json
```

Supports:

- IndexFlatL2
- IVFFlat

6. retriever.py

Purpose

Semantic retrieval from vector database.

Input:

query

Example query:

proprietary cryptographic hashing

Retrieves closest vectors.

Example result:

```
[
  {
    "text": "bcrypt hashing",
    "similarity_score": 0.89
  }
]
```

Uses:

- cosine similarity
- FAISS nearest neighbor search

7. legal_analyzer.py

Purpose

Compute legal/IP risk.

Produces:

- legal risk score
- risk level
- explanations

Risk Scoring Model

Active patent

+0.40

Commercial restriction

+0.30

US jurisdiction

+0.20

Competitor assignee

+0.10

Total score:

0.0 → 1.0

Risk Levels

LOW:

0 - 0.39

MEDIUM:

0.40 - 0.69

HIGH:

0.70 - 1.00

Example:

```
{
  "legal_risk_score": 0.9,
  "risk_level": "HIGH"
}
```

8. mhqg_engine.py

Purpose

Template-based Multi-Hop Question Generation.

Uses graph traversal.

Reasoning Path

```
Claim
↓
Dependency
↓
Patent Concept
```

3-hop reasoning detects hidden contradictions.

Example:

Claim:

```
We use proprietary encryption
```

Dependency:

```
bcrypt
```

Patent:

```
password hashing patent
```

Generated question:

You claim proprietary encryption, yet your architecture relies on bcrypt and overlaps with patented hashing mechanisms. How is your implementation differentiated?

Output

```
due_diligence_questions.json
```

Contains:

- question
 - risk score
 - risk level
 - audit trail
-

9. llm_mhqq.py

Purpose

LLM-based question generation using RAG.

Uses:

- FAISS retrieval
 - legal analysis
 - Ollama
 - Llama3
-

Pipeline

```
Claim
↓
Retriever
↓
Patent + dependency context
↓
Legal Analyzer
↓
Prompt construction
↓
Ollama Llama3
↓
Final question
```

Example Output

```
{
  "claim": "We use proprietary military-grade encryption",
  "risk_score": 0.91,
  "risk_level": "HIGH",
  "question": "Your system relies on open-source cryptographic libraries while overlapping with active patent claims. What protects your IP moat?"
}
```

Saved in:

```
llm_due_diligence_output.json
```

Execution Order

Run project in this order:

```
python src/extractors/whitepaper_parser.py
python src/extractors/github_parser.py
python src/extractors/patent_parser.py
python src/graph/knowledge_fusion.py
python src/vector/faiss_indexer.py
python src/reasoning/mhqc_engine.py
python src/reasoning/llm_mhqc.py
```

Final Outputs

due_diligence_questions.json

Contains:

- generated questions
- risk scores
- reasoning trail

llm_due_diligence_output.json

Contains:

- LLM generated questions
 - semantic evidence
 - legal reasoning
-

Core Novel Contributions

1. Cross-domain Knowledge Fusion

Combines:

- startup claims
 - codebase dependencies
 - patent knowledge
-

2. Multi-Hop Adversarial Reasoning

Reasoning path:

Claim → Dependency → Patent

Finds hidden contradictions.

3. Explainable Due Diligence AI

Each generated question includes:

- graph evidence
 - semantic evidence
 - legal evidence
-

Tech Stack

Programming:

- Python

Graph:

- NetworkX

Vector Search:

- FAISS

ML/NLP:

- Sentence Transformers
- MiniLM

Parsing:

- pdfplumber
- pypdf
- Tesseract OCR

LLM:

- Ollama
- Llama3
- Mistral